Noname manuscript No.
(will be inserted by the editor)

Beyond Black-Box Mutation: Grounding GPT-4.1 Mutant Generation in Historical Bug-Fix Patterns

Erlon Pereira Almeida · Daniel Lucredio · Rohit Gheyi · Pierre-Yves Schobbens · Gilles Perrouin · Paulo Anselmo da Mota Silveira Neto · Eduardo Santana de Almeida

Received: date / Accepted: date

Abstract Mutation testing is often constrained by the generation of unrealistic mutants, as existing approaches struggle to create faults that are both semantically meaningful and syntactically varied. Traditional and learning-based approaches alike struggle to create artificial faults that are both semantically meaningful and syntactically varied, often yielding simplistic or unstable results. This paper investigates using GPT-4.1 within a novel, two-stage workflow to generate mutants grounded in real fault patterns. We first systematically analyze multiple variations of prompt templates, combining five popular prompt techniques for the task of identifying code modification patterns at the Abstract Syntax Tree node level. Subsequently, we deploy the optimal prompt within a retrieval-augmented framework that dynamically matches and applies historical mutation operators to 8,322 real-world code snippets. Our findings reveal that different prompt strategies induce distinct "error profiles," effectively creating "specialists" in misusing specific Java nodes. Furthermore, compared to a specialized machine learning baseline, GPT-4.1 produces mu-

Erlon P. Almeida
Federal University of Bahia (UFBA), E-mail: erlon.almeida@ufba.br

Daniel Lucredio
Federal University of São Carlos (UFSCar), E-mail: dlucredio@gmail.com

Rohit Gheyi
Federal University of Campina Grande (UFCG), E-mail: rohitgueyi@gmail.com

Pierre-Yves Schobbens
University of Namur, E-mail: pierre-yves.schobbens@unamur.be

Gilles Perrouin
University of Namur, E-mail: gilles.perrouin@unamur.be

Paulo Silveira
Federal Rural University of Pernambuco (UFRPE), E-mail: pauloadmsn@gmail.com

Eduardo S. de Almeida
Federal University of Bahia (UFBA), E-mail: eduardo.almeida@ufba.br

tants with higher and more stable semantic resemblance to real faults, whereas the baseline excels in lexical fidelity but lacks stability. This work serves as a foundational investigation into the capabilities of LLMs for structured mutation testing.

Keywords GPT-4.1 · Mutation Testing · Prompt Engineering · Software Testing · LLM

1 Introduction

Mutation testing is a widely used technique for evaluating the fault-detection capability of test suites by introducing small, systematic changes called mutants to source code and measuring whether existing tests can detect them. These mutants are generated using mutation operators, such as *Delete Invocation* or *Update Binary Operator*, which simulate real faults and help assess the robustness of the test suite (Just et al., 2014; Papadakis et al., 2019b).

Despite its utility, mutation testing suffers from two well-known challenges. First, the selection and manual curation of mutation operators are time-consuming and often fail to capture the semantic behavior of real-world faults (Papadakis et al., 2018; Ojdanic et al., 2023a). The technique produces mutants in quantities several orders of magnitude greater than the number of lines in the program, thus necessitating multiple test executions (Gopinath et al., 2016, 2017b). For systems with millions of lines of code, this becomes computationally expensive (Gopinath et al., 2015). Beyond the issue of cost, a second well-known challenge is that traditional mutation approaches generate excessive redundant or equivalent mutants (Zhang et al., 2019; Wang et al., 2017, 2024a). Equivalent mutants are syntactically distinct but semantically identical to the original program, making them undetectable by any test case. Redundant mutants, while killable, are either (1) trivially detected by the same tests as other mutants or (2) subsumed by other mutants' behavior, offering no unique insight into test-suite effectiveness (Guimarães et al., 2020; Gheyi et al., 2021). The research community has pursued a wide array of strategies to mitigate these challenges, which can be broadly categorized into cost reduction and the management of equivalent mutants. Regarding cost, techniques range from selecting a minimal set of mutation operators (Mathur, 1991; Wong, 1993; Gopinath et al., 2017b) to randomly sampling from the full mutant set (Gopinath et al., 2017b; Papadakis et al., 2019a). For the issue of equivalence, solutions span from formal and heuristic detection methods (Offutt and Pan, 1996, 1997; Papadakis et al., 2015; Kintis et al., 2018) to prevention strategies and impact analysis (Grün et al., 2009; Schuler and Zeller, 2010; Papadakis et al., 2014; Wright et al., 2014; Yao et al., 2014). However, these approaches face inherent trade-offs: cost-reduction may compromise mutant representativeness, detection methods struggle with scalability, and prevention strategies introduce their own complexities.

These trade-offs reveal a fundamental tension within traditional mutation testing: the inherent limitations of syntactic, operator-driven approaches when

applied to the equivalent mutant problem (Kintis et al., 2016). Widely-used tools, such as MAJOR (Just, 2014), muJava (Ma et al., 2006), and PIT (Coles et al., 2016a) exemplify this paradigm, implementing selective sets of mutation operators to balance practical feasibility with fault-detection capability (Madeyski et al., 2014; Kintis et al., 2016; Gopinath et al., 2017a). While their design choices result in modest equivalent mutant rates (6.5% for PIT, 10.9% for muJava, and 11.6% for MAJOR) this is achieved through deliberate sacrifices in comprehensiveness rather than through intelligent reasoning about code semantics. Notably, only 4.1% of equivalent mutants are common across all three tools, underscoring that each tool's approach to operator selection produces fundamentally different mutant sets, with limited consensus on what constitutes an equivalent mutant (Just et al., 2012; Kintis et al., 2016). Higher-Order Mutation (HOM) strategies, which combine multiple first-order mutations to increase semantic diversity, offer a promising alternative that demonstrates significant reductions in equivalent mutants (Kurtz et al., 2016). Empirical evidence from implementations using the Judy tool (Madeyski and Radyk, 2010) shows that strategies like JudyDiffOp and Last2First achieve 82-88% and 75-82% reductions in equivalent mutants, respectively, while maintaining test effectiveness with only 1.75-4.2% loss in fault-detection capability (Madeyski et al., 2014). However, despite these advances, HOM strategies remain absent from the core of mainstream tools like muJava (Ma et al., 2006) and MAJOR (Just, 2014). Researchers (Laurent and Ventresque, 2019) have even attempted to extend the PIT framework for higher-order mutation in small-scale experiments (PIT-HOM), however, they explicitly concluded that without a sophisticated selection process, the resulting mutants are "largely redundant" (Laurent and Ventresque, 2019). Consequently, the core problem persists across both traditional and advanced mutation approaches: they remain fundamentally limited by their dependence on predefined syntactic rules and operator-based transformations, unable to reason about program context and semantics to prevent the generation of equivalent mutants in the first place.

Therefore, we posit that these persistent limitations stem from the absence of a cognitive component capable of intelligently selecting and combining mutation operators in a context-aware manner, tailored to the specific code under analysis. Strategies such as HOM attempted to address equivalence, but they are hindered by combinatorial explosion and often impair fault-detection capability (Nguyen and Madeyski, 2014; Wong et al., 2020). Similarly, tools that randomly select from a predefined operator set lack the semantic understanding to avoid nonsensical or equivalent changes.

On the other hand, the rapid evolution of Large Language Models (LLMs) such as GPT-4.1 has created new opportunities for leveraging these models across a wide range of software engineering activities. Recent studies demonstrate that LLMs can effectively support tasks such as code generation, summarization, fault localization, and mutation testing (Hou et al., 2024; Wang et al., 2024b). While studies have applied LLMs to mutation testing, these efforts typically rely on the model's ability to generate plausible code variants

in a black-box fashion (Degiovanni and Papadakis, 2022; Deng et al., 2023, 2024a). Such approaches often lack insights into whether the generated mutants meaningfully reflect real-world faults, or whether the underlying model has learned any interpretable patterns about how bugs are introduced and fixed.

In this paper, we present an exploratory study and proof of concept for a more transparent and structured use of LLMs in mutation analysis. We investigate a two-stage workflow that first prompts an LLM to identify complex modification patterns at the AST-node level from a dataset of real Java bug-fix commits, where code transformations naturally involve 1-16 concurrent modifications, effectively capturing historically-grounded higher-order mutations. These patterns are then reused as semantically-aware mutation operators to guide mutant generation. This approach moves beyond traditional black-box prompting by grounding the mutation process in the multi-operator change patterns characteristic of real fault fixes. A core component of our investigation involves a deep analysis of the error profiles, specifically, the node-level errors and their distribution, generated by different prompt templates. By systematically examining how the LLM behaves in this specific task, we aim to structure a reliable and interpretable method for mutation generation. Our work is motivated by the potential of appropriately prompted LLMs to generalize from noisy examples and recognize recurring, human-like code transformations (Brown et al., 2020; Wei et al., 2022, 2023; Hou et al., 2024; Wang et al., 2024b). Ultimately, by leveraging these capabilities, we seek to employ a cognitive component for generating higher-order mutants, with the long-term goal of reducing the generation of redundant and equivalent mutants, improving alignment with real-world defects, and enhancing the overall utility of mutation testing.

While recent work has begun to explore LLMs for generating mutants, such as intent-based mutation (Hamidi et al., 2025), these approaches focus on different generation strategies, such as mutating the natural language specifications (or "intents") of the code (Hamidi et al., 2025). In contrast, to the best of our knowledge, we are the first to: (1) systematically engineer and compare multiple prompt strategies to guide a state-of-the-art LLM (GPT-4.1) in extracting code modification patterns from real fixed-buggy commits at the AST-node level, and (2) reuse these historically-grounded patterns to guide a transparent mutant generation process. A core contribution of our work is the analysis of how prompt design induces distinct, specialized error profiles in the LLM, a phenomenon not investigated in prior work. To evaluate our method, we address two major research questions:

RQ1: What is the most effective way to instruct an LLM model in identifying modifications to Java code?

To answer RQ1, we conducted a quantitative analysis focused specifically on defective evolution (bug-fixes). We used an LLM on the main dataset of 8,322 Java fixed-buggy code pairs (Tufano et al., 2019) to assess its ability to identify the AST-node level modification patterns that transform fixed code back into its buggy version. Since LLM outputs are highly sensitive to prompt

formulation, we systematically evaluated four prompt templates, with different combinations of 5 popular prompt techniques (Few-Shot and Zero-shot Learning, Retrieval-Augmented Generation, Persona, Chain-of-Thoughts). We measured the correctness of the extracted AST node-level modifications and analyzed common biases and errors introduced by each prompt in each Java node.

The most effective prompting strategy from RQ1 was then adapted to guide the mutant generation workflow evaluated in RQ2, shifting its core task from operator identification to code generation.

RQ2: How effectively does LLM generate mutant code aligned with real faults in Java projects under the prompt strategies tested, compared to existing techniques?

Leveraging the best-performing prompt template from RQ1, we developed a mutant generation workflow that draws on a secondary dataset of 71,603 bug-fix examples to derive mutation operators. We then used these operators as a reference to generate mutant versions for the same main dataset used in RQ1. Our results demonstrate that the mutants generated by GPT-4.1 achieve structural characteristics with a level of semantic similarity comparable to those produced by state-of-the-art methods based on trained translator Retrieval Neural Network (RNN) models (Tufano et al., 2019). Furthermore, the advantage of our approach is particularly related to mutations involving modifications to deeply nested Java nodes, as measured by both semantic and lexical similarity metrics.

In summary, this study makes the following contributions:

- **An automated workflow** that leverages GPT-4.1 to generate mutants aligned with real-world Java faults, establishing the feasibility of using LLMs for this task.
- **The construction and empirical characterization** of a novel set of 72 mutation operators directly mined from 8,322 real bug-fix pairs using fine-grained AST analysis. These operators, encompassing 18 Java node types and 4 change operations, provide a structured, historically-grounded foundation for mutant generation that reflects the diversity of real fault patterns.
- **A comprehensive comparative analysis** of prompt engineering strategies, demonstrating that prompt choice does not lead to uniform improvement but instead induces distinct error profiles. This analysis reveals that each strategy systematically biases the model's performance, concentrating specific types of failure modes in a predictable manner.
- **Identification and categorization** of the specific Java node types where the LLM struggles the most. This reveals both chronically difficult nodes (e.g., TypeReference) and a fundamental confusion where key nodes (e.g., FieldRead) are frequently both omitted (False Negatives) and hallucinated (False Positives).

– **A publicly available replication package**[1], including all datasets, prompt templates, and evaluation scripts, to ensure the reproducibility of our findings and facilitate future research.

2 Related Work

Mutation testing is a well-established technique for evaluating test suites by deliberately introducing faults (mutants) into source code (Jia and Harman, 2011a; Papadakis et al., 2019a). Its effectiveness relies on the coupling hypothesis, which posits that mutants simulate real faults. This approach not only identifies test suite weaknesses but also supports critical tasks, such as fault localization (Gong et al., 2015; Du et al., 2024) and automated program repair (Weimer et al., 2013; Yi et al., 2018; Xiao et al., 2023).

While large-scale studies confirm a strong correlation between mutant detection and real bug discovery (Just et al., 2014; Laurent et al., 2022; Gay and Salahirad, 2023), a significant challenge persists. Traditional mutation often generates a high volume of redundant or invalid mutants, making the process computationally expensive and inefficient (Wang et al., 2024a). Consequently, generating mutants that are both representative of real faults and efficient to produce remains a crucial research objective.

2.1 Prior Mutation Testing Tools

Traditional mutation testing tools apply predefined operators to generate mutants, with implementations varying by programming language and the level of code representation (e.g., source code, AST, or bytecode) (Jia and Harman, 2011b). Well-known tools include Jumble (Irvine et al., 2007) and Pit (Coles et al., 2016b) for Java bytecode, MAJOR Just et al. (2011) for the Java AST, and AccMut (Wang et al., 2017) for LLVM IR in C programs.

A critical study by Gopinath et al. (2017a) compared tools like Major (Just et al., 2011) and Pit (Coles et al., 2016b), revealing that mutation scores are highly tool-dependent and rarely agree for the same project. This highlights a significant lack of standardization in the field, as the results are influenced more by the specific tool used than by a consistent measure of test suite quality.

We based on these traditional methods by avoiding the random selection of predefined rules that apply mutant operators uniformly across all code. Instead, our study investigates an LLM's capability to dynamically select context-aware mutant operators—validated against real developer-committed faults—by performing a semantic-based search on the secondary dataset to identify the most suitable mutations for each code scenario.

Shifting from predefined rules, Brown et al. (2017) empirically derived mutation operators by mining and reversing small, syntactic changes from real C code commits on GitHub. Their approach, implemented in the *mutgen* tool,

[1] `https://github.com/mutgen-gpt4-based/MutGen-LLM-Based-Research`

uses lexical analysis and pattern matching—not machine learning—to algorithmically harvest operators from code diffs, creating a large set of fault patterns grounded in actual developer behavior.

We fundamentally differ from the lexical, pattern-matching technique of Brown et al. (2017) by using GPT-4.1's semantic reasoning. Instead of manipulating tokens, our method employs a prompt template to guide an LLM in generating mutants, offering a more sophisticated alternative to purely syntactic operator application.

Machine learning (ML) has also been used to generate realistic mutants. Beller et al. (2021) built upon empirical principles with Mutation Monkey, a tool that learns mutation patterns from curated fault datasets (e.g., Defects4J) using a ML system called *Getafix*. Similarly, Tufano et al. (2019, 2020) employed RNN translation models to learn from code commit diffs, demonstrating the ability to generate mutants with high syntactic validity and a significant rate of exact matches to original bugs from software repositories.

In contrast to prior ML approaches that depend on a persistent operator catalog, our research proposes using prompt engineering with an LLM (without fine-tuning) to generate mutants competitively with state-of-the-art methods. Our key innovation is a dynamic workflow where mutation logic is generated on-the-fly in response to specific code inputs, rather than being statically retrieved from a pre-trained model.

2.2 LLM used in Mutation Testing

Recent studies have begun leveraging LLMs to address specific challenges in mutation testing, though not for the core task of mutant generation. Dakhel et al. (2024) used LLMs like Llama-2 and Codex to augment test generation for surviving mutants produced by a traditional tool (*MutPy*). Similarly, Tian et al. (2024) applied LLMs with optimized prompting to the difficult problem of equivalent mutant detection, showing significant improvement over traditional methods. In contrast to these works that use LLMs as complementary tools, our study positions the LLM as the core component for the direct generation of mutants.

Foster et al. (2025) developed a thematically similar multi-agent system using Llama3 70B. However, their work differs from ours in its fundamental objective and approach. Their system is concern-driven: it starts with a high-level issue (e.g., privacy) and uses mutant generation as an intermediate step to harden code by producing new unit tests. In contrast, our research is code-driven, focusing on using the LLM as the core engine for direct and general-purpose mutant generation, rather than for generating tests.

In contrast, even though both studies use agentic workflows, ours leverages mutation operators from historical bug-fix commits, grounding mutants in real-world faults. While Foster et al. (2025) focus on test synthesis, we emphasize a systematic prompt-engineering analysis, evaluating how prompting combinations affect GPT-4.1's ability to generate fault-representative mu-

tants—thereby shifting the optimization target from the tests to the mutation source itself.

Others studies have applied language models to mutant generation. Degiovanni and Papadakis (2022) used μBert for masked token replacement in Java, achieving a high fault detection rate. Expanding on this, Tip et al. (2025) compared multiple LLMs for generating JavaScript mutants, using AST node-based placeholders to enable more complex mutations than traditional token-level approaches. Our study diverges from prior approaches by leveraging GPT-4.1 to directly generate mutants from bug-fix datasets. Specifically, we rigorously analyze carefully designed prompts that explicitly instruct the LLM to select and apply mutation operators based on bug-fix history.

Li and Shin (2024) utilized mutation testing to evaluate LLM code understanding, specifically by mutating code descriptions to test for inconsistency detection. Our study diverges fundamentally from this approach in objective, methodology, and the LLM's role. While we extend the underlying principle of using mutation to probe model understanding, we apply it directly to the source code rather than natural language descriptions. Instead of positioning the model as a passive consistency checker, our systematic evaluation framework—grounded in 18 Java AST node types and 4 fundamental change operations—tasks the LLM as an active core generator and analyzer. Specifically, we assess GPT-4's ability to identify and replicate precise code modification patterns from bug-fix pairs, effectively reversing real-world fixes to recreate bugs.

Most closely related to our work, Wang et al. (2024a) conducted a large-scale benchmark of six LLMs against traditional (including muJava (Ma et al., 2006) and PIT (Coles et al., 2016a)) and learning-based mutation tools. Their evaluation uses an extensive array of metrics, including compilability rate and similarity to 851 real bugs, to assess the mutants. The LLMs are prompted using a direct "one-to-many" approach with general few-shot examples from the QuixBugs (Lin et al., 2017) dataset to generate diverse mutants for a given code snippet, producing 45 distinct AST node types compared to only 2 for rule-based tools (Wang et al., 2024a)

Our proposed research uses a "one-to-one" targeted replication method. For a given code snippet, our automated workflow searches in dataset (the secondary) to find a single, highly similar historical fault. The LLM's task is then to understand the specific transformation in that single example and replicate it on the target code. This historical grounding yields 72 empirically, derived mutation operators, providing a middle ground between rule-based approaches (2 AST node types) and pure LLM-based methods (45 types). In addition, our study explores how to architect an LLM-based system for a specific mutation task, whereas Wang et al. (2024a) focus on what the performance of different LLMs is on a more general mutation task.

3 Experiment Design

This study is grounded in the hypothesis that LLMs can serve as the primary component for generating significant mutants that accurately represent real-world faults. To validate this hypothesis, we conduct a deep, methodological investigation focused on a single, state-of-the-art model: GPT-4.1. This focused design choice is intentional and stems from our primary goal to perform a foundational characterization of the behavior and failure modes of a powerful LLM in the specific task of structured code mutation, rather than providing a superficial benchmark across multiple models.

The methodology proposed herein, grounding mutant generation in historical patterns, is implemented using the LangChain[2] framework, ensuring a modular architecture that is easily adaptable to virtually any LLM. While this technical design facilitates the seamless substitution of the underlying model, rigorously characterizing the generation phenomena requires an in-depth, controlled analysis of a single model's behavior across thousands of data points. Using a top-tier model serves a crucial methodological purpose: it ensures that any observed limitations are attributable to the inherent challenges of the task and our methodological choices, rather than to the model's fundamental incapacity. Consequently, if the proposed approach fails with a model of this caliber, it would indicate a fundamental limitation of the methodology itself. By establishing this detailed analytical framework with a high-performance baseline (GPT-4.1), we provide the essential groundwork for future research, allowing subsequent studies to systematically evaluate other LLMs (e.g., Code Llama[3] or Codex[4]) against a well-understood standard.

To operationalize this investigation, we developed an automated workflow that combines GPT-4.1 with historical bug-fix data to determine the optimal conditions for mutant generation while investigating the model's cognitive limits. The study is organized into two sequential steps:

Step 1 (Operator Identification): We tested the LLM's ability to identify unique modification patterns from defective evolution (bug-fixes) in Java code at the Abstract Syntax Tree (AST) node level. This was accomplished by systematically evaluating various prompting techniques, ranging from simple instructions to robust strategies involving Retrieval-Augmented Generation (RAG) (Soudani et al., 2024). To provide a granular analysis of the model's performance in this task, we addressed two specific sub-questions:

- **RQ1.1:** To what extent does our prompt engineering strategy dictate the model's error profile?
- **RQ1.2:** Which Java node types are difficult for GPT-4.1 to identify accurately, given the prompt strategies applied in this study?

This initial evaluation was a crucial prerequisite for the generation task. Given the high sensitivity of LLMs to prompt construction, the approach's ef-

[2] `https://www.langchain.com`
[3] `https://codellama.dev/about`
[4] `https://openai.com/pt-BR/index/codex-now-generally-available`

fectiveness can vary drastically based on the chosen strategy. Since the specific impact of different prompt techniques on this structured task was unknown, answering these sub-questions provided the essential insights needed to select an optimal prompt. This selection, in turn, enabled the creation of an effective workflow for Step 2, aimed at maximizing the model's reasoning capabilities to ensure the high fidelity and semantic representativeness of the generated mutants.

Step 2 (Mutant Generation): We applied the optimal prompt template from Step 1, adapting its core instruction to shift the task from operator identification to code generation, to generate mutants for the entire main dataset. We then compared them with those produced by a validated baseline tool based on Recurrent Neural Network (RNN) translation models (Tufano et al., 2019). This baseline choice was deliberate, enabling a direct comparison between two distinct machine learning paradigms. Notably, this design contrasts a general-purpose LLM (GPT-4.1), guided only by prompts, against specialist RNN models specifically trained and optimized for this dataset (Tufano et al., 2019). This comparison is methodologically the most appropriate, as the abstracted, non-compilable nature of the dataset makes a direct evaluation against traditional tools (like PIT (Coles et al., 2016b) or MAJOR (Just et al., 2011)) infeasible. The comparison utilized two metrics: the BLEU score for lexical realism and semantic similarity (via embedding vectors) to assess conceptual equivalence. This process revealed the strengths and limitations of our approach in using GPT-4.1 for mutation testing. We detail the specific methodology for each step below.

3.1 Dataset

The dataset chosen for our study is in Java language and is unconventional. This dataset underwent a generalization process for each node, which forces the LLM to understand the code's structure, as its presentation prevents confusion with internal data that was incessantly used for training or fine-tuning. This makes it perfect for our experiment of exploring the limits of a language model's ability to comprehend and manipulate Java code.

This dataset, developed by Tufano et al. (2019), was created from 787,178 Java bug-fix commits mined from public GitHub repositories between 2011 and 2017. The central idea of their work was to automatically learn how to generate realistic mutants by training Recurrent Neural Network (RNN) models to reverse the transformations present in the bug fixes (i.e., learning how to transform fixed into buggy code). The bug-fix commits were initially identified from the GitHub Archive using keyword-based filters (e.g., "fix" and "bug") and the Google BigQuery APIs. From this set, method-level code changes were extracted using the GitHub Compare API[5] and an AST based differentiation tool, GumTree (Falleri et al., 2014). Several filters were applied to ensure data

[5] `https://docs.github.com/pt/rest/commits/commits?apiVersion=2022-11-28`

quality. First, the scope of the analysis was restricted exclusively to commits from Java-centric repositories. From this initial set, we further excluded commits that modified more than five Java files—as these likely represent tangled changes. The final dataset consists of method-level "*Transformation Pairs*" (TPs), each containing a buggy method and its corresponding fixed version.

To reduce vocabulary complexity and focus the models' learning on structural transformations, Tufano et al. (2019) applied a preprocessing pipeline that abstracts Java source code. In this process, project-specific identifiers are generalized to preserve structural patterns while removing contextual details. For instance, as shown in Figure 1, type names such as `Integer` and `ListManager` were replaced with generic placeholders such as `TYPE_1` and `TYPE_2`, while method names like `getMinElement` were abstracted to `METHOD_1`. Crucially, common Java keywords (e.g., `List`, `if`, `null`) and operators were preserved to maintain the fundamental syntactic structure and semantic context of the code constructs. A fundamental step in their process was the preservation of 272 highly frequent terms, or "idioms" (such as equals or the literal 0), which were identified through statistical analysis and manual curation to retain important semantic context (Tufano et al., 2019).

This process, originally designed to train RNN mutation models, resulted in a dataset partitioned into a training set of 82,515 TPs and a test set of 10,313 TPs. To allow for a more granular analysis, Tufano et al. (2019) organized the dataset using an unsupervised clustering algorithm. This method resulted in five distinct clusters, grouping Transformation Pairs that share similar structural characteristics based on the nature of the original code modifications. For our experiments, we utilize the established test set (Tufano et al., 2019), drawing upon the data from all five clusters to underpin our LLM-based analysis.

```
public Integer getMinElement(List myList) {
    if(myList.size() >= 1) {
        return ListManager.min(myList);
    }
    return null;
}
```

```
public TYPE_1 METHOD_1 ( List VAR_1 ) { if (
VAR_1 . size ( ) >= 1 ) { return TYPE_2 . min (
VAR_1 ) ; } return null ; }
```

Fig. 1 Code abstraction example from Tufano et al. (2020)

Many Java nodes occur with extremely low frequency, presenting two challenges for our GPT-4.1 analysis. First, it is difficult to draw meaningful conclusions about the model's performance on these infrequent nodes. Second, these rare cases introduce noise when analyzing TPs characterized by more frequent nodes. For instance, Cluster 1 is predominantly defined by *Invocation* modi-

fications, which appear 3,945 times. Conversely, nodes such as *while*, *assert*, *ArrayTypeReference*, and *break* appear only 6, 6, 5, and 2 times, respectively.

To address this challenge, we determined that a data pruning step was necessary. Based on an analysis of the node distribution within each cluster, using a static code diff tool, we established a minimum threshold of 100 occurrences for a given Java node type to be included in our experiment. Consequently, any Transformation Pair composed exclusively of node types that fell below this threshold was removed from the dataset. This pruning approach served the dual purpose of eliminating statistically insignificant data points and reducing the computational costs associated with the LLM's API, while still retaining a sufficient data volume for a robust analysis.

A key characteristic of the dataset is that each Transformation Pair is typically composed of multiple, fine-grained modification in each Java nodes. This compositional nature has a direct implication on the filtering process: removing a TP to filter out one low-frequency Java node inherently removes all other nodes bundled within that same TP, thus altering the overall frequency distribution.

Consequently, our filtering process did not eliminate every instance of rare components, as some were preserved when they co-occurred in a TP with high-frequency Java nodes that met the threshold. Nevertheless, the pruning approach was successful in its primary goal: to drastically reduce the variety of Java nodes modified and noise in the data for a more focused analysis.

Our final curated dataset (the main dataset) is a subset of test dataset from Tufano et al. (2019). This subset contains 8,322 TPs covered by modifications in 18 distinct Java node components. The final distribution of these nodes across the five distinct clusters is presented in Table 1 and analyzed in the discussion that follows.

Table 1 Absolute and Relative Frequency (%) of Components by Cluster.

Component	**Cluster 1**	**Cluster 2**	**Cluster 3**	**Cluster 4**	**Cluster 5**
Assignment	—	—	—	115 (1.14%)	**926 (21.51%)**
FieldWrite	—	—	—	—	**912 (21.18%)**
If	236 (2.25%)	73 (2.17%)	72 (1.29%)	**837 (8.29%)**	113 (2.62%)
BinaryOperator	389 (3.72%)	132 (3.93%)	290 (5.21%)	**1400 (13.87%)**	111 (2.58%)
Invocation	**3434 (32.83%)**	**735 (21.87%)**	706 (12.69%)	**2415 (23.92%)**	440 (10.22%)
TypeAccess	**3490 (33.37%)**	426 (12.67%)	**636 (11.43%)**	951 (9.42%)	348 (8.08%)
FieldRead	901 (8.61%)	169 (5.03%)	389 (6.99%)	1054 (10.44%)	283 (6.57%)
VariableRead	674 (6.44%)	**564 (16.78%)**	**827 (14.87%)**	454 (4.50%)	153 (3.55%)
ThisAccess	305 (2.92%)	—	149 (2.68%)	102 (1.01%)	205 (4.76%)
Literal	551 (5.27%)	142 (4.22%)	**706 (12.69%)**	846 (8.38%)	524 (12.17%)
ConstructorCall	—	128 (3.81%)	196 (3.52%)	59 (0.58%)	131 (3.04%)
LocalVariable	—	**398 (11.84%)**	—	—	—
Parameter	—	—	450 (8.09%)	63 (0.62%)	—
TypeReference	58 (0.55%)	**450 (13.39%)**	607 (10.91%)	178 (1.76%)	16 (0.37%)
Wra	—	—	306 (5.50%)	—	—
UnaryOperator	—	—	—	221 (2.19%)	—
Return	135 (1.29%)	59 (1.76%)	145 (2.61%)	420 (4.16%)	—
Block	285 (2.72%)	85 (2.53%)	83 (1.49%)	981 (9.72%)	143 (3.32%)
Total	**10,458**	**3,361**	**5,562**	**10,096**	**4,305**
Modifications per TP	2–6	6–12	1–3	3–8	3–8
Amount TP per cluster	2692	415	2490	1922	803
Primary Role	Domain Operations	Object Initialization	Methods and Constructors	Control Flow Logic	State Management

Cluster 1, the largest cluster with 10,458 modifications, exhibits a profile specialized in domain operations. It is characterized by the dominance of modifications to *Invocation* (32.83%) and *TypeAccess* (33.37%) nodes, indicating a primary focus on executing business logic through method and class calls. This is supported by data-reading operations (*FieldRead*: 8.61%, *VariableRead*: 6.44%). The absence of control-flow node modifications suggests predominantly sequential, linear execution logic.

Cluster 2 (3,361 modifications) exhibits a profile that aligns with object initialization and configuration. This is suggested by a high concentration of nodes involved in invoking functions and handling types. The most prominent activity is method invocation (*Invocation*, 21.87%), supported by operations involving types (*TypeAccess*, 12.67% and *TypeReference*, 13.39%). The process appears to rely on reading data from variables (*VariableRead*, 16.78%) and using local variables (*LocalVariable*, 11.84%), which likely supply the necessary parameters and temporary values for the initialization logic. The notably low presence of control flow nodes like *If* (2.17%) and *BinaryOperator* (3.93%) indicates these modifications are likely focused on the linear setup and assembly of objects rather than on implementing complex conditional logic. The collective evidence points to a cluster specialized in the preparatory stages of creating and configuring objects.

Cluster 3 (5,562 modifications) is characterized by the definition and modification of methods and constructors. This is indicated by a significant presence of *Parameter* nodes (8.09%), which are exclusive to signatures. The internal logic of these units involves *VariableRead* (14.87%), *Invocation* (12.69%), and *Literal* (12.69%) nodes, suggesting operations that read data and call other methods, often with constants. The modifications are granular, averaging 2 to 4 node changes. Therefore, the profile of this cluster suggests that it predominantly represents modifications to reusable code units like methods and constructors.

Cluster 4 (10,096 modifications) is characterized by modifications that strongly suggest a focus on altering a program's control flow and decision-making logic. This is primarily evidenced by the high co-occurrence of *If* (8.29%) and *BinaryOperator* (13.87%) nodes, which are the core components of conditional statements. The conditions within these structures appear to be based on reading object state, as indicated by *FieldRead* (10.44%), and compared against fixed values, with textitLiteral (8.38%) nodes. The significant presence of *Block* (9.72%) nodes confirms that the changes involve the bodies of code executed by these control flow structures. While method calls are involved (*Invocation* with 23.92%), the overall profile of this cluster is dominated by the components that define and branch execution paths.

Cluster 5 (4,305 modifications) exhibits a profile that is highly suggestive of a specialization in object state management. This interpretation is primarily driven by the dominant co-occurrence of *Assignment* (21.51%) and *FieldWrite* (21.18%) nodes, a combination that typically represents operations that update field values. The notable prevalence of *Literal* (12.17%) and *ThisAccess* (4.76%) nodes further supports the notion that these modifications often in-

volve assigning constant values to an object's own fields. Therefore, the collective evidence implies that this cluster predominantly captures the fundamental operations for defining and updating an object's internal data.

3.2 Step 1: Analyzing LLM's Capabilities for Detecting AST Nodes.

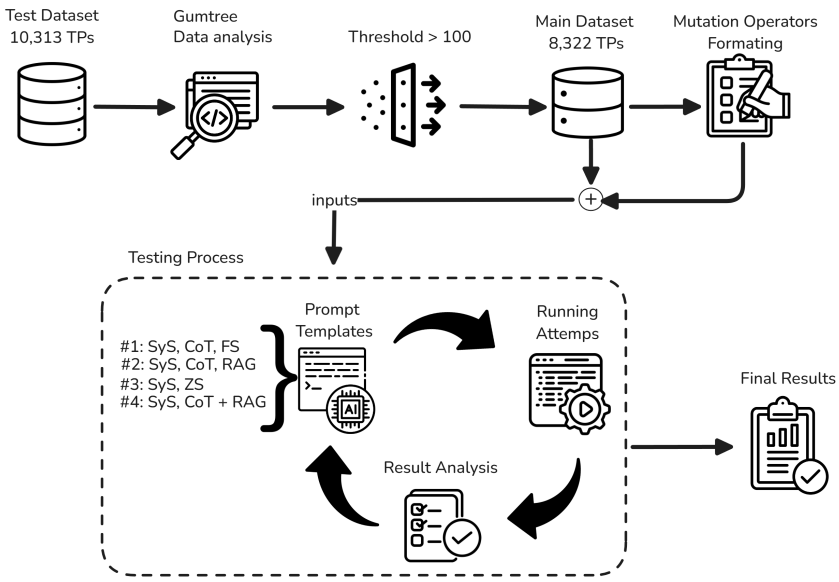

Fig. 2 Workflow done in the process of testing GPT-4.1 on identifying modifications in each TP.

Figure 2 depicts the workflow used to evaluate GPT-4.1's ability to identify code modification patterns that transform fixed versions into buggy ones. The process began with an analysis of a dataset containing 10,313 TPs. This analysis revealed a substantial imbalance in the frequency of approximately 48 types of Java code nodes. These nodes could be modified through four operations: *Insert*, *Move*, *Update*, and *Delete*. We define a mutation operator as one of these four operations applied to a specific Java node type. As a result of the imbalance, all TPs associated with node types that occurred fewer than 100 times were excluded, resulting in a refined dataset of 8,322 TPs spanning 18 Java nodes. Considering those four operators by each node, we got a total of 72 mutation operators.

To identify the mutation operators in the dataset, we generated AST-level differences using the GumTree[6] tool (Falleri et al., 2014; Martinez et al., 2023), an algorithm that computes fine-grained and accurate source code differences,

[6] `https://github.com/SpoonLabs/gumtree-spoon-ast-diff`

configured with the Spoon parser. This general approach of using GumTree for code differencing has been previously validated in studies such as Tufano et al. Tufano et al. (2019), which allowed us to use the generated outputs as a reliable ground truth for assessing whether GPT-4.1 could identify the same transformations using GumTree's notation.

The evaluation was conducted over four iterative rounds (or "Attempts"), each leveraging a distinct prompt template that incorporated the techniques outlined in Table 2. In each attempt, GPT-4.1's raised-mutation operators were compared against the Gumtree (Falleri et al., 2014; Martinez et al., 2023) results.

Upon completing this analysis, we compiled: (1) mutant operators identified as true positives, false positives and false negatives, (2) the accuracy rate for correctly assessed TPs within each cluster, and (3) the percentage of Java nodes that generated accurate responses, (4) and the most repeated errors raised.

Table 2 Overview of All Prompting Techniques Used.

Technique	**Description**
Few-shot learning (FS)	Offering the model one or more input-output examples to guide text generation (Brown et al., 2020).
Retrieval-Augmented Generation (RAG)	Enhancing the model's responses by retrieving relevant information from external knowledge sources or documents, which are then used as context to generate more accurate and informed outputs (Soudani et al., 2024).
Zero-shot learning (ZS)	Providing the model with direct task instructions without prior examples (Wei et al., 2022).
Chain of Thought (CoT)	Decomposing complex problems into a sequence of intermediate steps for more detailed and interpretable problem-solving (Wei et al., 2023).
Persona (SyS)	Assigning a specific role or persona to the language model to influence output style and content (White et al., 2023).

While absolute error counts (FPs and FNs) are useful for understanding the overall volume of errors, they can be biased by the frequency of node types in the dataset. To measure the model's inherent difficulty with each node type, independent of its frequency, we define a Normalized Failure Index (NFI). The NFI is calculated for each node type and each prompting strategy (Attempt) as the ratio of the total number of TPs with an error (either FP or FN) to the total frequency of that node's modifications in the ground truth dataset:

$$\text{NFI} = \frac{(\text{Total TPs with FP}) + (\text{Total TPs with FN})}{\text{Total Frequency of the Node Modification}} \times 100\% \qquad (1)$$

This index provides a measure of the model's proportional unreliability, allowing for a fair comparison between common and rare node types.

Thus, the objectives of this evaluation are to:

1. Quantify the deviation of GPT-4.1's results from the baseline tool;
2. Identify the most effective approach for employing an LLM in mutation operator identification, and hence, for mutation generation;
3. Assess the reliability of the model's outputs; and
4. Examine patterns and potential biases in its results.

It is important to note that while several prompt techniques are reused across the four templates, their specific content is not always identical. For example, the *Persona* technique (White et al., 2023) is used in all four templates with largely consistent content (act as an "expert in Abstract Syntax Tree (AST) analysis for the Java language"). Conversely, the *Chain-of-Thought* technique (Wei et al., 2023), although employed in three of the four templates, is slightly adapted in each case to complement the other techniques present within the same prompt template.

Attempt 1: We employed three prompt techniques. The *Persona* (White et al., 2023) technique to configure GPT-4.1 as an "expert in Abstract Syntax Tree (AST) analysis for the Java language", the *Chain-of-Thought* (Wei et al., 2023) technique to guide step-by-step analysis of each TPs, and *Few-Shot learning* (Brown et al., 2020), where we provided six exemplar fixed-to-buggy transformations from the training dataset of Tufano et al. (Tufano et al., 2019), following the number of few-shot examples proposed by Deng et al. (Deng et al., 2024b) (six examples), which they found to be a sufficient number of examples for an LLM to act in code task. To mitigate GPT-4.1's tendency for verbose outputs, we included a curated list of all 18 Java nodes in the prompt, while expecting the model to propose valid modifications to any input TP. Figure 3 shows the templates's structure used in this Attempt 1.

The advantages of this approach lie in its ability to effectively guide the LLM's code analysis and standardize its output. The primary disadvantages are the extensive prompt size, which makes it the most costly strategy, and the large volume of information supplied to the model, which can introduce noise and adversely affect the quality of the generated response.

Attempt 2: This attempt enhances the prompt by incorporating a RAG approach (Soudani et al., 2024). We created a knowledge base containing the explanations for all 18 annotated Java nodes (the same across all Attempts). This text was converted into vector embeddings using OpenAI's `Text-Embedding-3-Large` model[7] and stored in a vector database to enable semantic retrieval. For each TP, relevant chunks from the knowledge base were retrieved and provided to GPT-4.1. Additionally, the *Chain-of-Thought* (CoT) technique (Wei et al., 2023) was employed to instruct the model to perform a step-by-step analysis.

This RAG-based methodology is designed to ground the model's reasoning in the provided documentation, thereby preventing hallucinations or reliance

[7] `https://platform.openai.com/docs/models/text-embedding-3-large`

Prompt Template from Attempt 1

System (Persona)

You are an expert in Abstract Syntax Tree (AST) analysis for the Java language.

Context (Zero-Shot)

You will be provided with two code snippets, "snippet one" (the fixed version) and "snippet two" (the buggy version).
These snippets are a processed representation of Java code where identifiers have been replaced with placeholders and IDs.
Your task is to identify the precise AST modifications required to transform "snippet one" into "snippet two". The modifications fall into four categories: "Insert", "Delete", "Update", or "Move". You must associate each modification with a specific Java node component from the official list provided below.
* <*List_of_all_Java_nodes_and_their_definitions*>

Instructions (CoT)

Follow these steps to generate your final answer:

1. Perform a structural comparison between "snippet one" and "snippet two".
2. Identify all AST nodes that were added ("Insert"), removed ("Delete"), moved from its original place to another ("Move"), or had their value changed ("Update").
3. For each modification, identify the type of the AST node involved and match it to a component from the **Official Java AST Component List**.
4. Format your final answer as a single line, enclosed in square brackets. Each entry must follow the "<Operation> <ComponentType>" format. Separate multiple entries with a semicolon.

Examples (FS)

Six-shots examples are provided for your reference:
* <*List_of_six_TPs_followed_by_mutation_operators_from_Gumtree*>

Fig. 3 The prompt structure from Attempt 1.

on potentially incorrect knowledge from its internal training. The prompt explicitly directs the model to prioritize the retrieved context, ensuring that its analysis is based on the specific operator definitions supplied. In next Attempt, we formulated a method that represents the most straightforward approach for formulating a mutation operator identification task. The advantage of next Attempt lies in its ability to evaluate whether GPT-4.1 can effectively apply the correct modification from a mutation operator, given that the correct answer is contained within a provided set of options.

Attempt 3: We simplified GPT-4.1's decision-making by reformulating the task as a multiple-choice problem. This time, we provided a curated set of mutation operators that always included the correct solution.

The multiple-choice format followed these rules:

- For single-operator transformations: 5 candidate operators; and
- For each additional required operator: +2 candidate operators (e.g., 8 options for 2-operator transformations).

We used *Zero-shot* (Wei et al., 2022) learning to establish the task and directed GPT-4.1 to choose from the candidate operators while operating in *Persona* (White et al., 2023) consistency as previous attempts.

This setup allows for an objective assessment of the model's capability to correlate the input with the potential correct answers. However, this approach is limit to identification task, as we had to provide the set of mutation operators that simulates the correct fault from buggy version. Next Attempt we designed a template to leverage the full reasoning potential of GPT-4.1. Its initial, unstructured responses were subsequently processed and aligned with the data in the vector store.

Attempt 4: We employed a two-stage process per TP to identify the mutation operators. The first stage involved tasking GPT-4.1 to independently evaluate the TP without contextual information from the vector database, thereby collecting suggestions based on its internal knowledge. In the second stage, we performed a semantic search on GPT-4.1's free-text response to identify which nodes in the vector database it referred to. That is, for each modification proposed by GPT-4.1, we subsequently standardized it by performing RAG. This was achieved by finding the chunk in the vector database that had the highest semantic similarity to the proposed modification. The Persona (White et al., 2023), Zero-Shot (Wei et al., 2022), and Chain-of-Thought (Wei et al., 2023) prompting techniques remained the same as in previous Attempts.

The advantage is the reduced size of the required prompt. The main disadvantage, however, lies in the difficulty of standardizing GPT-4's output, as it must be mapped to the structured data from the vector database. Performing an accurate semantic search for this mapping can be complex.

3.3 Step 2: Generating Mutant Code

The second Step of this study consists of a workflow to instruct an LLM (GPT-4.1) to generate mutants that represent real software faults. The process aims to transform fixed versions from the main dataset into mutant versions that are similar to their original buggy counterparts.

To achieve this, the primary objective is to define the mutation operators that will guide GPT-4.1. These operators are extracted from TPs in the secondary dataset, which is a subset of the train dataset used by Tufano et al. (2019) for training RNN models.

As shown in Figure 4, the workflow begins with data preparation (Figure 4-block A). The main dataset (Figure 4-A2) contains 8,322 fixed codes for which mutant versions need to be generated. The mutation operators for each fixed code are extracted from TPs in the secondary dataset (Figure 4-A1). This dataset was filtered using the GumTree tool (Falleri et al., 2014; Martinez et al., 2023), following the node filtering used in the main dataset (Section 3.1), to retain only TPs whose modifications involve the 18 most frequent Java node types shown in Table1, providing us with a search base comprising 71,603 TPs. The selection of these filtered TPs is based on the semantic similarity

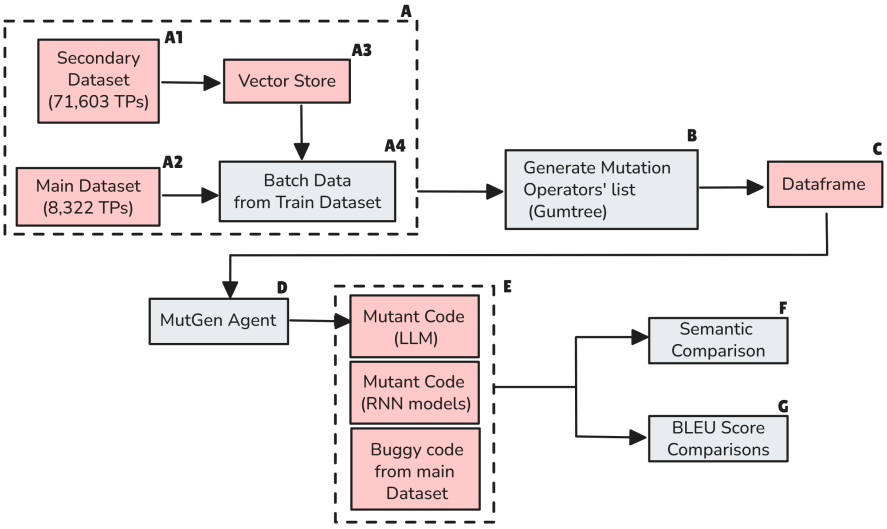

Fig. 4 Multi-agent workflow built for Step 2

between the fixed code from the main dataset and the codes from the secondary dataset. To optimize this large-scale search, the secondary dataset was indexed into a vector database (Figure 4-A3), enabling efficient retrieval of similar TPs through an Approximate Nearest Neighbor (ANN) (Andoni et al., 2018; Malkov and Yashunin, 2018; Jafari et al., 2021) search algorithm.

After selecting similar TPs for each fixed code in the main dataset, the mutation operators contained within them were extracted and formatted (Figure 4-B). This information was then consolidated into a DataFrame (Figure 4-C), which served as input for a specialized LLM agent called MutGen (Figure 4-D). For this task, the GPT-4.1 input prompt was adapted from the template that achieved the best performance in RQ1 (operator identification) to suit the mutant generation task. Due to the I/O-bound nature of the task and to comply with the rate limits imposed by the GPT-4.1 API, the generation of the 8,322 mutants was not executed all at once. Instead, we adopted a batch processing strategy. The process was conducted on a machine equipped with a 13th-generation Intel Core i7 processor, 32 GB of RAM, and a GPU with 16 GB of VRAM. For each batch, 15 concurrent requests were sent to the API to maximize throughput without exceeding the service limits.

At the conclusion of the Step 2 workflow, the mutant versions generated by the LLM and those from Tufano et al. (2019) (RNN models) (Figure 4-E) were used to perform a lexical (Figure 4-G) and semantic (Figure 4-F) comparison against the buggy versions from the main dataset. The baseline models from Tufano et al. (2019) are based on an RNN Encoder-Decoder architecture. Crucially, this baseline is not a single, general model, but an ensemble of five specialized models ($M_1 ... M_5$). As described in the original study, each of these models was trained and fine-tuned exclusively on the data from its

corresponding cluster. The training for each model was substantial, utilizing cluster sizes of $C_1 = 30,385$ TPs, $C_2 = 7,016$, $C_3 = 29,625$, $C_4 = 25,320$, and $C_5 = 10,798$. This establishes a fundamental asymmetry in our comparative analysis: the performance of a single, general-purpose foundation model (GPT-4.1), guided only by a prompt, is benchmarked against five specialist models, each evaluated on the specific data category on which it was exclusively trained.

To assess lexical similarity, we employed the BLEU (Bilingual Evaluation Understudy) metric (Papineni et al., 2002). BLEU is an automated score designed to quantify the lexical similarity between a machine-generated text and human-written reference texts. Its core mechanism involves comparing sequences of words (n-grams) to measure overlap, while also adjusting for word frequency and penalizing overly short generated texts. The final BLEU score, ranging from 0 to 1, provides an indicator of the lexical fidelity of the generated text, showing high correlation with human judgments of quality. In our study, the mutants (generated by both the LLM and the RNN) were compared against the buggy versions from the main dataset, which served as the reference.

To assess semantic resemblance, we employed Cosine Similarity, a metric that quantifies the similarity between two texts by measuring the cosine of the angle between their vector representations (embeddings). The process first transforms each code snippet, both the generated mutant and the reference buggy code, into a numerical vector using *OpenAI's text-embedding-3-large* model. Unlike metrics such as Euclidean distance that measure magnitude, cosine similarity focuses purely on the orientation of these vectors in a high-dimensional space. The resulting score ranges from -1 to 1, where a value of 1 indicates identical orientation (maximum semantic similarity) and 0 indicates orthogonality (no similarity). This score served as our metric for evaluating whether a generated mutant preserved the functional purpose of the original fault.).

This study evaluates generated mutants against reference buggy code along two complementary dimensions: lexical fidelity and semantic resemblance. Lexical fidelity, measured using the BLEU score, quantifies the surface-level n-gram overlap between the generated and reference code, assessing the correct use of keywords, identifiers, and syntactic structure. Semantic resemblance, measured via Cosine Similarity of code embeddings, evaluates whether the mutant tries to preserve the functional purpose and logical intent of the original, even with differing syntax. Employing both metrics allows us to distinguish merely syntactic copies from conceptually equivalent mutants, providing a holistic evaluation of the models' generation capabilities.

4 Results

This section presents the findings obtained through the methodology used to answer the research questions.

RQ1: What is the most effective way to instruct an LLM model in identifying modifications to Java code?

Table 3 presents the results of all Attempts for each cluster in Step 1. These results consider the complete response, meaning we verified whether the mutant operators provided by the LLM (specifying which modification was made to which node) are included in the list identified by GumTree (Falleri et al., 2014; Martinez et al., 2023).

We observed that the examples from the Few-Shot technique influenced the accuracy results, showing that Attempt 1 achieved balanced results, ranging from 70.57% to 74.08% across all clusters. We also noted that the Recall results were the highest among all Attempts, reaching up to 61.36% in Cluster 3.

In Attempt 2, although GPT-4.1 showed significant precision results in clusters 1 and 2, its performance dropped drastically in the remaining clusters, falling to 42.29% in cluster 5. Furthermore, it presented the worst Recall results among all Attempts, with all values below 28%, reaching as low as 12.29%, also in cluster 5.

Attempt 3 produced the best precision results among all Attempts. This is due to the fact that this prompt template provides the correct mutant operators for each TP within the operator set. The lowest precision result was in cluster 3, with 79.21%, and the highest was in cluster 5, with 90.53%. The Recall results all remained below 32%, which lowered the F1-Score level.

The results of Attempt 4 showed precision slightly lower than that of Attempt 3, notably in clusters 1, 2, and 5, in addition to being the Attempt with

Table 3 Classification metrics from GPT-4.1 Responses per each Attempt.

	Cluster	Precision	Recall	F1-Score
Attempt 1	1	73.06	57.19	64.16
	2	74.08	53.73	62.29
	3	70.57	61.36	65.64
	4	74.21	40.04	52.01
	5	74.59	51.89	61.21
Attempt 2	1	84.42	27.52	41.51
	2	80.56	24.16	37.17
	3	48.94	29.79	37.04
	4	54.11	16.37	25.13
	5	42.29	12.29	19.04
Attempt 3	1	90.24	31.46	46.65
	2	81.14	26.75	40.23
	3	79.21	17.73	59.57
	4	83.03	26.17	39.80
	5	90.53	30.20	45.29
Attempt 4	1	86.24	27.98	42.25
	2	75.02	22.79	34.96
	3	69.07	43.47	53.36
	4	75.44	24.37	36.83
	5	89.68	24.41	38.38

the smallest prompt size. Furthermore, Attempt 4 presented slightly higher Recall results than Attempt 3, reaching 43.47% in cluster 3.

Table 4 Aggregated Results of Precision, Recall, and F1-Score by Approach.

	Precision (%)	**Recall (%)**	**F1 (%)**
Attempt 1	73.302	**52.842**	**61.062**
Attempt 2	62.064	22.026	31.978
Attempt 3	**84.830**	26.462	46.308
Attempt 4	79.090	28.604	41.156

To determine the most effective prompt template for the mutant operator identification task, a comparative analysis of the aggregated results was conducted, as presented in Table 4. Attempt 1, which combined techniques such as Chain-of-Thought (CoT) with 6-shot learning examples, demonstrated the best overall performance, achieving the highest F1-Score of 61.062%. This result was primarily driven by its Recall of 52.842%, the highest among all approaches, indicating a superior ability to identify the majority of relevant mutation operators. Although Attempt 3 achieved the highest Precision (84.830), its low Recall (26.462%) resulted in a considerably lower F1-Score. This suggests that, despite their accurate identifications, both Attempt 3 and Attempt 4 failed to detect a large portion of the necessary transformations.

Finding 1: The prompt template from Attempt 1 (Fig. 3) provides the best balance between Precision and Recall among all Attempts and is the most effective for the mutant operator identification task.

RQ1.1: To what extent does our prompt engineering strategy dictate the model's error profile?

To answer RQ1.1, we conducted a quantitative analysis to evaluate whether GPT-4.1 adequately selected the node types to be modified within each TP. Our error analysis was therefore focused on the correctness of the node choice itself, rather than the frequency of its application within the code. The errors were categorized into two types: False Positives (FPs), where the model chose a node type that was not part of the ground truth transformation, and False Negatives (FNs), where the model failed to choose a node type that was required for the transformation. An error for a given node type was counted once per TP, treating each TP as an independent test of the model's selection capability. This approach prevents complex TPs with multiple instances of the same node from disproportionately influencing the results. We then aggregated these TP-level error counts across all four prompting strategies (Attempts) to identify the nodes with the highest absolute error volumes.

Our analysis of False Positives revealed that a small subset of node types accounts for the majority of hallucinated content. As detailed in Table 5, the five node types with the highest volume of FPs were *VariableRead* (1968 instances), *Invocation* (1301), *Return* (1280), *Literal* (1006), and *FieldRead*

Table 5 Percentage Contribution of Each Prompting Strategy to FP Errors per Node Type.

Node Type	Attempt 1	Attempt 2	Attempt 3	Attempt 4	Total Errors
VariableRead	68.09%	22.56%	8.13%	1.22%	1968
Invocation	4.84%	48.81%	4.92%	41.43%	1301
Return	6.41%	10.39%	10.16%	73.05%	1280
Literal	87.77%	0.40%	8.25%	3.58%	1006
FieldRead	94.62%	0.00%	5.38%	0.00%	893
LocalVariable	1.12%	78.52%	2.36%	18.00%	889
Assignment	11.56%	20.62%	9.83%	58.00%	519
Parameter	12.16%	23.08%	11.66%	53.10%	403
TypeAccess	11.84%	83.88%	1.51%	2.77%	397
If	5.76%	15.45%	5.50%	73.30%	382
TypeReference	49.60%	25.47%	10.72%	14.21%	373
Block	33.72%	12.21%	22.09%	31.98%	172
ConstructorCall	12.88%	0.61%	15.34%	71.17%	163
UnaryOperator	3.39%	2.54%	6.78%	87.29%	118
Wra	7.46%	77.61%	1.49%	13.43%	67
BinaryOperator	23.88%	22.39%	28.36%	25.37%	67
ThisAccess	50.00%	2.63%	18.42%	28.95%	38
FieldWrite	61.29%	9.68%	22.58%	6.45%	31

(893). However, a deeper analysis of the error distribution, shown in Table 5, demonstrates that these high volumes are not uniform but are instead concentrated in specific failure modes for each prompt strategy. For instance, 68.09% of *VariableRead* errors and 94.62% of *FieldRead* errors originated from Attempt 1, whereas 73.05% of *Return* errors were attributed to Attempt 4. On the other hand, errors for *BinaryOperator* were almost evenly distributed across all four attempts (ranging from 22.39% to 28.36%), suggesting a more fundamental, strategy-agnostic difficulty for the model. It indicates that the choice of prompt technique often induces a specific error profile, effectively creating specialists in raising certain types of incorrect nodes.

Table 6 Percentage Contribution of Each Prompting Strategy to FN Errors per Node Type.

Node Type	Attempt 1	Attempt 2	Attempt 3	Attempt 4	Total Errors
TypeAccess	19.19%	24.84%	27.17%	28.80%	11713
FieldRead	18.22%	28.10%	25.56%	28.12%	7622
Literal	8.03%	36.28%	23.82%	31.87%	6348
VariableRead	14.97%	23.42%	28.09%	33.52%	6259
Invocation	16.21%	11.54%	26.79%	45.45%	4972
Block	26.21%	20.25%	25.74%	27.81%	4682
BinaryOperator	12.99%	34.69%	20.81%	31.52%	4258
TypeReference	16.65%	27.81%	24.29%	31.24%	3207
ThisAccess	8.01%	38.36%	22.75%	30.88%	1697
FieldWrite	44.04%	0.00%	0.00%	55.96%	1401
Assignment	20.05%	48.34%	14.34%	17.27%	1297
If	0.81%	98.14%	0.65%	0.41%	1234
ConstructorCall	9.70%	44.17%	15.77%	30.36%	1021
Return	19.76%	22.71%	23.88%	33.65%	850
UnaryOperator	0.00%	35.73%	28.99%	35.28%	445
Parameter	1.15%	87.36%	1.15%	10.34%	87
LocalVariable	62.50%	18.75%	12.50%	6.25%	16
Wra	0.00%	0.00%	100.00%	0.00%	4

Subsequently, we applied the same analytical process to the False Negatives (FNs), which represent modifications the model failed to identify. The analysis revealed that the volume of FN errors was substantially higher than that of FPs, a finding consistent with the model's overall low Recall (Table 4). The five most frequently omitted node types, as detailed in Table 6, were *TypeAccess* (11713 instances), *FieldRead* (7622), *Literal* (6348), *VariableRead* (6259), and *Invocation* (4972).

Following the same analysis from the FP Table 5, two primary patterns emerged. First, a set of nodes exhibited a consistent error pattern, with omissions being relatively distributed across all four attempts. Nodes such as *TypeAccess*, *FieldRead*, and *Block* fall into this category. For *TypeAccess*, for instance, each strategy contributed between 19.2% and 28.8% of the total errors. This indicates a persistent challenge for the model in identifying these types of modifications, regardless of the prompt design.

The second pattern revealed highly concentrated, strategy-specific failure modes. The most striking example is the *If* node, where a single strategy, Attempt 2, was responsible for an overwhelming 98.1% of all omissions. A similar pattern was observed for *Parameter*, where Attempt 2 also accounted for 87.4% of the errors. This demonstrates that certain prompts create critical blind spots for the model, making it a specialist in omitting specific node types.

Finding 2: The prompt engineering strategy is a dominant factor in dictating the model's error profile. Different strategies were found to induce highly distinct and specialized failures modes, often leading to opposing patterns of False Positives and False Negatives and effectively creating error specialists for specific node types.

RQ1.2: Which Java node types are difficult for GPT-4.1 to identify accurately, given the prompt strategies applied in this study?

While the previous analysis demonstrated that each prompt strategy induces a distinct error profile, a direct comparison of error volumes can be influenced by the frequency with which each node type appears in the dataset. To definitively answer RQ1.1—"To what extent does our prompt engineering strategy dictate the model's error profile?"—and to determine the model's inherent difficulty with each node, a normalized analysis is required.

To this end, we calculated a Normalized Failure Index (NFI) for each node across all four attempts. This index, as defined in the methodology section, represents the ratio of total errors (FPs + FNs) generated by an attempt for a specific node type to the total frequency of that node in the ground truth dataset.

Table 7 presents the results of this analysis, enabling a fair comparison of the model's proportional struggle across different node types. The analysis of this table reveals two primary categories of difficulty: (1) nodes that pose a challenge, exhibiting high error rates across all strategies; and (2) nodes that suffer from catastrophic failures, with extremely high error rates under specific prompt strategies.

Table 7 Normalized Failure Index (NFI) per Node Type and Prompting Strategy (Corrected).

Nós	Total	Attempt 1 (%)	Attempt 2 (%)	Attempt 3 (%)	Attempt 4 (%)
assignment	1041	30.74%	70.51%	22.77%	50.43%
BinaryOperator	2322	24.50%	64.25%	38.98%	58.53%
Block	**1577**	**81.48%**	**61.45%**	**78.82%**	**86.05%**
ConstructorCall	514	23.50%	87.94%	36.19%	82.88%
FieldRead	**2796**	**79.90%**	**76.61%**	**71.39%**	**76.65%**
if	1331	2.40%	95.42%	2.18%	21.41%
Invocation	7730	11.24%	15.64%	18.06%	36.21%
Literal	**2769**	**50.31%**	**83.32%**	**57.60%**	**74.36%**
LocalVariable	398	5.03%	176.13%	5.78%	40.45%
Parameter	513	9.75%	32.94%	9.36%	43.47%
Return	759	32.94%	42.95%	43.87%	160.87%
ThisAccess	761	20.37%	85.68%	51.64%	70.30%
TypeAccess	5851	39.22%	55.41%	54.50%	57.84%
TypeReference	**1309**	**54.93%**	**75.40%**	**62.57%**	**80.60%**
UnaryOperator	221	1.81%	73.30%	61.99%	117.65%
fieldwrite	912	69.74%	0.33%	0.77%	86.18%
VariableRead	**2672**	**85.22%**	**71.48%**	**71.78%**	**79.42%**
Wra	306	1.63%	16.99%	1.63%	2.94%

Finding 3: The most problematic nodes are those with high proportional error rates across all prompt strategies, such as TypeReference, FieldRead, and VariableRead. This indicates a fundamental difficulty in processing these node types, which is distinct from high-volume errors (like Invocation) that are primarily a function of their frequency in the dataset.

RQ2: How effectively does GPT-4.1 generate mutant code aligned with real faults in Java projects under the prompt strategies tested, compared to existing techniques?

Figures 5 and 6 present a comparative analysis between two fundamentally different approaches: a single, general-purpose LLM (GPT-4.1), guided only by a prompt across all five bug-fix clusters, and the five specialized RNN models from Tufano et al. (2019), where each model is an expert trained on a single, corresponding cluster. Figure 5 shows semantic similarity scores, where GPT-4.1 demonstrates a remarkably stable and consistent performance. This stability indicates that the model is less susceptible to the particular nature of the modifications within each cluster, suggesting a strong generalization capability. In contrast, Figure 6 presents BLEU scores for lexical similarity, measuring the token-level overlap between the generated and reference code.

Our workflow generated mutants for all 8,322 TPs at a rate of 0.7418 mutations per second, with a computational cost of approximately USD 0.004545 per generated mutant code.

The analysis of semantic similarity scores, presented in Figure 5, reveals that GPT-4.1 consistently generates code with higher and more stable semantic alignment to the buggy reference code. In contrast, the RNNs-based model exhibits significantly greater variability and a higher propensity for low-score outliers across most clusters.

This pattern of superior stability from GPT-4.1 is evident across the different code patterns represented by the clusters. In Clusters 1 and 2, for ex-

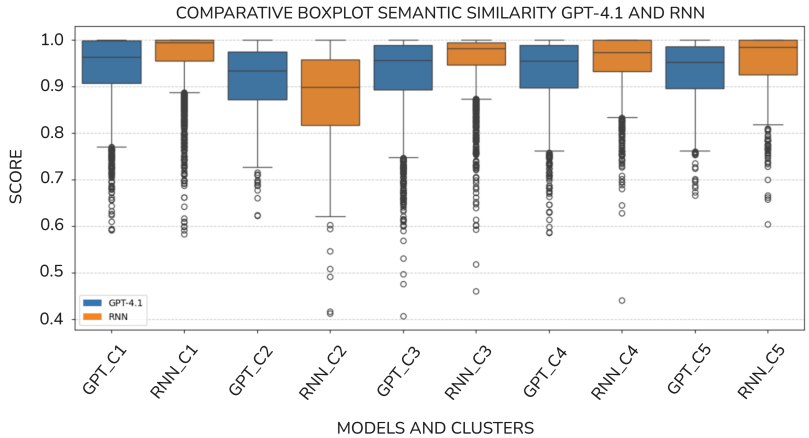

Fig. 5 Workflow done in the process of testing GPT-4.1 on identifying modifications in each TP.

ample, while both models achieved high median scores, the RNNs' performance showed a much wider dispersion. Even in Cluster 3, where the median scores were comparable, GPT-4.1 demonstrated greater robustness by producing fewer low-similarity outliers. Cluster 4 highlighted a notable trade-off: the RNN models achieved slightly higher peak scores, yet this came at the cost of greater instability, producing several outputs with substantial semantic deviation (below 0.82). GPT-4.1, by contrast, maintained its characteristic high consistency. A notable exception is Cluster 5, representing simpler bug-fix structures, where both models performed exceptionally well, suggesting both approaches are effective for less complex transformations. The overall results indicate that while both models are capable of generating semantically similar code, GPT-4.1 does so with a much lower outliers range.

The analysis of lexical similarity, measured via the BLEU score, is presented in Figure 6. The results reveal a clear trade-off between the two models. The RNN-based models generally achieved higher median BLEU scores across four of the five clusters (C1, C3, C4, C5), indicating a stronger literal reproduction of token sequences from the original buggy code.

However, this higher lexical alignment from the RNN came at the cost of significantly lower consistency, a pattern particularly evident in the first three clusters. In Clusters 1, 2, and 3, the RNN's performance was marked by a wider score dispersion and a substantial number of severe low-score outliers. In contrast, in Clusters 4 and 5, the RNN's results were much more stable and compact, similar to GPT-4.1's. Across the board, GPT-4.1 demonstrated superior in predictable results (stable), with a more compact distribution of scores and a near-total absence of the catastrophic low-score outliers that affected the RNN.

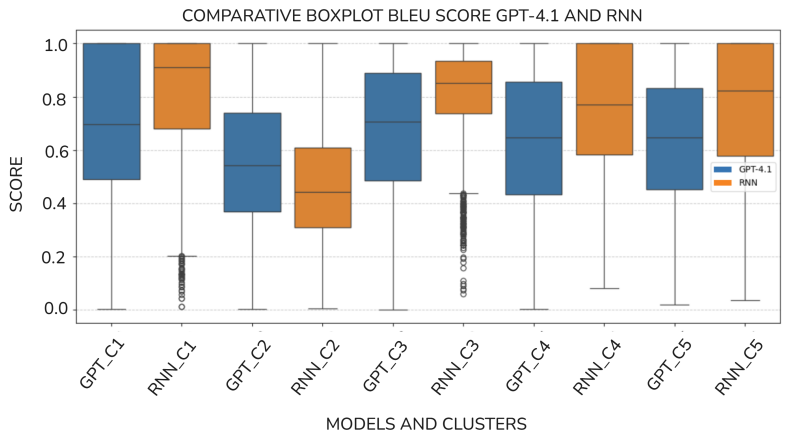

Fig. 6 Workflow done in the process of testing GPT-4.1 on identifying modifications in each TP.

Finding 4: GPT-4.1 and the specialized RNN models baseline exhibit a clear trade-off in mutant generation. GPT-4.1 produces mutants with higher Semantic Similarity and, crucially, demonstrates a performance that is less susceptible to the specific nature of the modifications within each cluster. The RNN model achieves better Lexical similarity (BLEU score) but with higher output variability and more frequent semantically divergent mutants. GPT-4.1 emerges as the more robust and generalizable approach for generating meaningful, fault-aligned mutants.

5 Discussion

In this section, we interpret the findings presented in the previous section and discuss their broader implications. Our results revealed two central themes: (1) the profound impact of prompt engineering on the model's error behavior in the node identification task, leading to the creation of distinct "error profiles"; and (2) a clear performance trade-off between GPT-4.1 and the RNN baseline in the mutant generation task, highlighting a dichotomy between semantic and lexical similarity.

5.1 Prompt Strategy as an Error Profile Director

The finding that prompt engineering strategies induce distinct and predictable error profiles offers a significant contribution beyond simply ranking prompt performance. It reframes the challenge of prompt optimization, transforming an apparent limitation into a design opportunity for both academic research and industrial applications.

For Academic Research, this discovery challenges the conventional methodology of evaluating prompts based on single, aggregated performance metrics like F1-score. We demonstrate that a deeper analysis of error profiles is crucial for a complete understanding of a prompt's behavior. Our results suggest that future research should focus not on finding a single "perfect prompt"—as our findings suggest this may be unattainable—but rather on exploring prompt ensembles. The existence of these error specialists implies that combining the outputs of multiple, complementary prompts—each strong where others are weak—could lead to a far more robust and comprehensive result than any individual prompt could achieve. This opens a new avenue for research in creating more reliable LLM-based code analysis tools.

For Industrial Applications, this insight provides a direct path toward building more reliable and predictable AI systems. Instead of relying on a single prompt, a production system could implement a "committee of experts" architecture. Multiple LLM instances, each guided by a specialized prompt (e.g., one tailored for control flow, another for data access), could analyze code in parallel. A voting or merging mechanism could then consolidate the outputs, effectively mitigating the individual "blind spots" of each prompt and increasing the system's overall accuracy and robustness. Furthermore, understanding a prompt's specific error profile allows for intelligent risk management. In a context where errors in control flow (*If* nodes) are more critical than errors in constant values (*Literal* nodes), developers can strategically choose the prompt that minimizes the most costly type of error, even if it is not the best-performing prompt overall. Additionally, the detailed error analysis provides a crucial roadmap for future model specialization through fine-tuning. By identifying the specific node types where the general-purpose model struggles, a targeted dataset of these challenging examples can be created to fine-tune smaller, more efficient models, producing highly specialized and cost-effective tools optimized to avoid the specific failure modes identified in this study.

5.2 The Trade-off Between Lexical and Semantic Similarity

The comparative analysis of lexical and semantic similarity between single, general-purpose LLM (GPT-4.1) against five specialized RNN models, provides significant contributions to both academic research and industrial practice.

For Academic Research, our findings highlight the limitations of relying solely on surface-level metrics like the BLEU score for evaluating modern LLMs in mutant generation tasks. The specialized RNN models, while achieving higher median BLEU scores, proved to be both more sensitive to the specific modification patterns of each cluster and more prone to instability, as evidenced by their frequent generation of severe low-score outliers. This demonstrates that academic benchmarks must increasingly incorporate semantic evaluation metrics to capture a model's true capability for generat-

ing functionally correct and meaningful code, rather than its ability for mere pattern replication. Furthermore, this study provides a clear characterization of the different operational philosophies of these models: the RNN acts as a high-fidelity pattern reproducer, which excels at literal imitation but is brittle when patterns diverge. In contrast, GPT-4.1 operates as a semantic generator, which paraphrases and abstracts intent, leading to lower lexical scores but higher semantic coherence and robustness.

This distinction is critically contextualized by recent research that directly investigates the link between lexical and semantic similarity in mutation testing. The study by Ojdanic et al. (2023b) empirically demonstrated that syntactic similarity is not a reliable proxy for semantic similarity, finding that mutants with high BLEU scores often failed to replicate the semantic behavior of real faults. Crucially, their work evaluated four fault-seeding tools against the Defects4J v2.0 benchmark, and the tool with the worst performance was DeepMutation—which is based on the same RNN translation model architecture (Tufano et al., 2019) as our baseline. Despite being optimized for syntactic mimicry, its mutants semantically resembled only 9.76% of the real faults, a result significantly lower than the other three evaluated techniques (which scored between 61.39% and 76.44%).

For industrial applications, this distinction offers a practical framework for model selection. Regarding tasks requiring high reliability, predictability, and maintainability—where the generated code must be easily understood and trusted by human developers—GPT-4.1's approach is demonstrably superior. Its high semantic similarity and its robustness against producing less outliers make it a more suitable model for production environments. Our results quantify the risk associated with models such as the RNN, which, despite achieving performance peaks, can introduce unpredictable and nonsensical outputs into an automated pipeline. This argues for prioritizing stable and semantically coherent models like GPT-4.1, even at the expense of perfect lexical replication.

Finally, our comparative analysis provides a framework for evaluating generative models based on the critical trade-off between lexical and semantic similarity. This guides both future research and enables a more strategic selection of these technologies, allowing practitioners to choose the model that best fits their specific project needs.

6 Threats to Validity

We identify a few threats to the validity of our study, which are categorized next. For each threat, we also discuss the mitigation strategies employed.

Internal Validity: The findings of our study are closely tied to the specific instructions and prompting strategies employed with the LLM. During the initial exploratory phase, we systematically experimented with combinations of prompting techniques, contextualization strategies, and RAG (Soudani et al., 2024) to assess the LLM's ability to identify code modification patterns. We also investigated how many prompt variations were needed to capture each

TP adequately. Furthermore, we explored multiple ways to translate these identified patterns into mutant operators to generate meaningful mutant code. We selected and reported the four most promising approaches highlighting the rationale behind each attempt.

Furthermore, we acknowledge that the stochastic nature of LLMs can introduce random effects. To mitigate this, we deliberately configured the model's temperature parameter to 0, effectively disabling stochastic nucleus sampling (top-p) and ensuring the most deterministic outputs possible. Regarding experimental repetition, while traditional protocols for stochastic models often prescribe multiple runs to establish statistical robustness, replicating the full generation process for 8,322 TPs multiple times was infeasible due to the substantial computational costs and strict API rate limits associated with the commercial GPT-4.1 model. However, we argue that the sheer scale of our dataset (over 8,000 samples) provides sufficient statistical power to stabilize aggregate metrics (such as F1-score and Semantic Similarity) and identify systematic patterns that would be unlikely to emerge from random fluctuations alone, thereby mitigating the potential impact of individual stochastic outliers.

External Validity: The generalizability of our findings is primarily limited by the scope of our dataset and methodology. First, this study focuses solely on Java projects. Although the proposed methodology is adaptable to other programming languages, the performance of the LLM and the specific error patterns observed may not directly translate to languages with different syntactic and semantic structures. Second, our analysis is based on a curated set of 18 mutant operators. While these operators were selected to be representative of the 8,322 Transformation Pairs in main dataset, they do not constitute an exhaustive list of all possible mutation patterns found in prior work (Tufano et al., 2019).

Third, the generalizability of our results is bounded by the use of a single LLM (GPT-4.1). While the proposed two-stage workflow is a model-agnostic methodology, the quantitative performance, the relative effectiveness of the prompt templates, and the specific "error profiles" we identified are intrinsically linked to the capabilities and architectural biases of this particular model. These behaviors might differ significantly in other LLMs (e.g., open-source models or those with different training data), and thus, our study establishes a foundational baseline with a state-of-the-art model rather than a universal benchmark.

Furthermore, a significant threat to external validity arises from the unconventional nature of our dataset. While the dataset is large, which supports the generalizability of our findings, the Transformation Pairs were intentionally abstracted and generalized. This design forces the LLM to perform a purely structural analysis, identifying modification patterns without the rich semantic context of a full project (e.g., variable meanings, method purposes, or overall architecture). We hypothesize that this represents a more challenging, "worst-case" scenario for the LLM. Consequently, the performance metrics reported in this study should be interpreted as a conservative lower bound; it is plausible that in a real-world industrial application, where the LLM would have

access to the full semantic context of the code, its performance in identifying and generating modifications would be substantially higher.

Construction validity: A threat to construct validity concerns the interpretation of the model's low recall, particularly in the context of False Negatives. Our methodology correctly and objectively measures the model's output against the exhaustive, fine-grained syntactic modifications identified by the baseline tool. However, the interpretation of these results must account for the inherent operational philosophy of LLMs like GPT-4.1, which often prioritize semantic compression.

Rather than generating a complete list of all syntactically affected nodes, the model may provide a single, high-level description of a change, implicitly considering lower-level nodes as inferable. This behavior inevitably leads to a higher number of measured FNs when compared to our systematic, exhaustive baseline. Consequently, the low Recall reported in our study should not be interpreted solely as a lack of capability, but rather as evidence of a fundamental mismatch between the LLM's semantically-driven, summary-oriented approach and the exhaustive, syntactically-driven nature of our ground truth. Acknowledging this difference is crucial for a fair interpretation of the model's performance.

A significant threat to construct validity is the lack of operational validation for the generated mutants. Due to the abstract nature of the dataset's Transformation Pairs (which use generic placeholders like `TYPE_1` or even `METHOD_1`), the mutants generated by both GPT-4.1 and the RNN baseline are not directly compilable or executable. Consequently, we were unable to perform dynamic analysis to calculate a traditional mutation score (i.e., determine if the mutants are killed by a test suite) or compare LLM's results with other tool such as Pit (Coles et al., 2016b) or LEAM (Tian et al., 2023). Our evaluation of effectiveness is therefore necessarily limited to the lexical and semantic alignment of the generated code with the reference fault, not its concrete operational behavior when executed. A critical direction for future work is therefore to bridge this gap by adapting the developed workflow to operate on a benchmark of real Java code, such as Defects4J. This would enable a direct comparison of the model's performance against other state-of-the-art tools on a standard benchmark and validate its practical effectiveness in an end-to-end scenario.

7 Conclusion

This paper presents a foundational investigation into the operational characteristics of GPT-4.1 on the key tasks of identifying and generating mutants. The goal was not to propose a production-ready tool, but to establish a comprehensive benchmark against a specialized, RNN-based approach. By benchmarking its performance, our objective was to map the LLM's operational characteristics, its limitations, and the fundamental trade-offs involved. Our findings therefore reveal a complex and nuanced performance landscape,

moving beyond simple metrics of success or failure to provide a clearer understanding of how to effectively leverage these models in practice.

The primary conclusion of this work is that the efficacy of LLMs in code analysis is not monolithic but is characterized by a series of significant trade-offs. First, in the node identification task, we found that prompt engineering acts as a bias director, inducing specialized and often contradictory error profiles rather than providing uniform improvement. This suggests that the search for a single, universally optimal prompt is likely futile. Our analysis of the Normalized Failure Index (NFI) further pinpointed the model's deepest struggles, distinguishing between nodes that pose a chronic, strategy-agnostic challenge (e.g., TypeReference) and those that suffer from catastrophic failures under specific prompts (e.g., If).

Second, in the mutant generation task, we identified a clear trade-off between GPT-4.1 and an RNN baseline. GPT-4.1 excels in generating mutants with superior semantic similarity, consistently producing meaningful and predictable code. The RNN, conversely, is better at achieving high lexical similarity through literal reproduction but at the cost of significant instability and a propensity for generating low-quality, nonsensical outliers.

The findings of this study open several critical directions for future research. The most pressing question arising from our work is whether the error profile phenomenon—where prompt strategies induce distinct and specialized failure modes—is a behavior unique to GPT-4.1 or a general characteristic of large language models. Therefore, our primary future work will be to replicate this methodology with other state-of-the-art LLMs (such as models from Anthropic, Llama, Deepseek, Gemini families) to investigate if prompt engineering consistently acts as a bias director across different model architectures.

Subsequently, building on our error specialist finding, we plan to develop and evaluate a prompt ensemble system. By combining the outputs of multiple, complementary prompts, we hypothesize that we can create a more robust and comprehensive result than any single prompt could achieve. Furthermore, we intend to bridge our generative framework with traditional mutation testing tools. This will allow us to evaluate the efficacy of existing test suites against our realistic, LLM-generated mutants and compare their "kill rate" against the simpler, synthetic mutants generated by conventional tools. Finally, to validate our approach in a full-scale industrial setting, we will apply our methodology to an end-to-end real-world project, allowing us to assess performance with full semantic context.

Author Contributions Erlon Pereira Almeida: Conceptualization, Methodology, Code Implementation, Validation, Formal analysis, Investigation, Data Curation, Writing - original draft, Visualization.
Daniel Lucredio: Conceptualization, Methodology, Formal analysis, Resources, Writing - review & editing, Supervision, Project administration.
Rohit Gheyi: Conceptualization, Methodology, Formal analysis, Writing - review & editing, Supervision.
Pierre-Yves Schobbens: Methodology, Formal analysis, Validation, Writing - review & editing.
Gilles Perrouin: Methodology, Formal analysis, Validation, Writing - review & editing.

Paulo Anselmo da Mota Silveira Neto: Methodology, Formal analysis, Validation, Investigation, Writing - review & editing.

Eduardo Santana de Almeida: Conceptualization, Methodology, Formal analysis, Resources, Writing - review & editing, Supervision, Funding acquisition.

Data Availability The datasets generated during and/or analysed during the current study are available in the GitHub repository: `https://github.com/mutgen-gpt4-based/MutGen-LLM-Based-Research`

Funding The authors did not receive support from any organization for the submitted work.

Declarations

Ethical approval Not applicable.

Informed consent Not applicable.

Conflict of Interest The authors have no relevant financial or non-financial interests to disclose. The authors declare that they have no conflict of interest.

Clinical Trial Number Clinical trial number: not applicable.

References

Andoni A, Indyk P, Razenshteyn I (2018) Approximate nearest neighbor search in high dimensions. URL `https://arxiv.org/abs/1806.09823`, `1806.09823`

Beller M, Wong CP, Bader J, Scott A, Machalica M, Chandra S, Meijer E (2021) What it would take to use mutation testing in industry: a study at facebook. In: Proceedings of the 43rd International Conference on Software Engineering: Software Engineering in Practice, IEEE Press, ICSE-SEIP '21, p 268–277, DOI 10.1109/ICSE-SEIP52600.2021.00036, URL `https://doi.org/10.1109/ICSE-SEIP52600.2021.00036`

Brown DB, Vaughn M, Liblit B, Reps T (2017) The care and feeding of wild-caught mutants. In: Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, Association for Computing Machinery, New York, NY, USA, ESEC/FSE 2017, p 511–522, DOI 10.1145/3106237.3106280, URL `https://doi.org/10.1145/3106237.3106280`

Brown TB, Mann B, Ryder N, Subbiah M, Kaplan J, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A, Agarwal S, Herbert-Voss A, Krueger G, Henighan T, Child R, Ramesh A, Ziegler DM, Wu J, Winter C, Hesse C, Chen M, Sigler E, Litwin M, Gray S, Chess B, Clark J, Berner C, McCandlish S, Radford A, Sutskever I, Amodei D (2020) Language models are few-shot learners. URL `https://arxiv.org/abs/2005.14165`, `2005.14165`

Coles H, Laurent T, Henard C, Papadakis M, Ventresque A (2016a) Pit: a practical mutation testing tool for java. In: Proceedings of the 25th international symposium on software testing and analysis, pp 449–452

Coles H, Laurent T, Henard C, Papadakis M, Ventresque A (2016b) Pit: a practical mutation testing tool for java (demo). In: Proceedings of the 25th International Symposium on Software Testing and Analysis, Association for Computing Machinery, New York, NY, USA, ISSTA 2016, p 449–452, DOI 10.1145/2931037.2948707, URL https://doi.org/10.1145/2931037.2948707

Dakhel AM, Nikanjam A, Majdinasab V, Khomh F, Desmarais MC (2024) Effective test generation using pre-trained large language models and mutation testing. Information and Software Technology 171:107468, DOI https://doi.org/10.1016/j.infsof.2024.107468, URL https://www.sciencedirect.com/science/article/pii/S0950584924000739

Degiovanni R, Papadakis M (2022) µbert: Mutation testing using pre-trained language models. In: 2022 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), pp 160–169, DOI 10.1109/ICSTW55395.2022.00039

Deng Y, Xia CS, Peng H, Yang C, Zhang L (2023) Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, Association for Computing Machinery, New York, NY, USA, ISSTA 2023, p 423–435, DOI 10.1145/3597926.3598067, URL https://doi.org/10.1145/3597926.3598067

Deng Y, Xia CS, Yang C, Zhang SD, Yang S, Zhang L (2024a) Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, Association for Computing Machinery, New York, NY, USA, ICSE '24, DOI 10.1145/3597503.3623343, URL https://doi.org/10.1145/3597503.3623343

Deng Y, Xia CS, Yang C, Zhang SD, Yang S, Zhang L (2024b) Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries. In: Proceedings of the 46th IEEE/ACM international conference on software engineering, pp 1–13

Du B, Han B, Liu H, Chang Z, Liu Y, Chen X (2024) Neural-mbfl: Improving mutation-based fault localization by neural mutation. In: 2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC), pp 1274–1283, DOI 10.1109/COMPSAC61105.2024.00168

Falleri JR, Morandat F, Blanc X, Martinez M, Monperrus M (2014) Fine-grained and accurate source code differencing. In: Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering, Association for Computing Machinery, New York, NY, USA, ASE '14, p 313–324, DOI 10.1145/2642937.2642982, URL https://doi.org/10.1145/2642937.2642982

Foster C, Gulati A, Harman M, Harper I, Mao K, Ritchey J, Robert H, Sengupta S (2025) Mutation-guided llm-based test generation at meta. URL

https://arxiv.org/abs/2501.12862, 2501.12862

Gay G, Salahirad A (2023) How closely are common mutation operators coupled to real faults? In: 2023 IEEE Conference on Software Testing, Verification and Validation (ICST), pp 129–140, DOI 10.1109/ICST57152.2023.00021

Gheyi R, Ribeiro M, Souza B, Guimarães M, Fernandes L, d'Amorim M, Alves V, Teixeira L, Fonseca B (2021) Identifying method-level mutation subsumption relations using z3. Information and Software Technology 132:106496, DOI https://doi.org/10.1016/j.infsof.2020.106496, URL https://www.sciencedirect.com/science/article/pii/S095058492030238X

Gong P, Zhao R, Li Z (2015) Faster mutation-based fault localization with a novel mutation execution strategy. In: 2015 IEEE Eighth International Conference on Software Testing, Verification and Validation Workshops (ICSTW), pp 1–10, DOI 10.1109/ICSTW.2015.7107448

Gopinath R, Alipour A, Ahmed I, Jensen C, Groce A (2015) How hard does mutation analysis have to be, anyway? In: 2015 IEEE 26th International Symposium on Software Reliability Engineering (ISSRE), pp 216–227, DOI 10.1109/ISSRE.2015.7381815

Gopinath R, Alipour MA, Ahmed I, Jensen C, Groce A (2016) On the limits of mutation reduction strategies. In: 2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE), pp 511–522

Gopinath R, Ahmed I, Alipour MA, Jensen C, Groce A (2017a) Does choice of mutation tool matter? Software Quality Journal 25:871–920

Gopinath R, Ahmed I, Alipour MA, Jensen C, Groce A (2017b) Mutation reduction strategies considered harmful. IEEE Transactions on Reliability 66(3):854–874, DOI 10.1109/TR.2017.2705662

Grün BJ, Schuler D, Zeller A (2009) The impact of equivalent mutants. In: 2009 International Conference on Software Testing, Verification, and Validation Workshops, IEEE, pp 192–199

Guimarães MA, Fernandes L, Ribeiro M, d'Amorim M, Gheyi R (2020) Optimizing mutation testing by discovering dynamic mutant subsumption relations. In: 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST), pp 198–208, DOI 10.1109/ICST46399.2020.00029

Hamidi A, Khanfir A, Papadakis M (2025) Intent-based mutation testing: From naturally written programming intents to mutants. In: 2025 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), pp 347–357, DOI 10.1109/ICSTW64639.2025.10962508

Hou X, Zhao Y, Liu Y, Yang Z, Wang K, Li L, Luo X, Lo D, Grundy J, Wang H (2024) Large language models for software engineering: A systematic literature review. ACM Trans Softw Eng Methodol 33(8), DOI 10.1145/3695988, URL https://doi.org/10.1145/3695988

Irvine SA, Pavlinic T, Trigg L, Cleary JG, Inglis S, Utting M (2007) Jumble java byte code to measure the effectiveness of unit tests. In: Testing: Academic and Industrial Conference Practice and Research Tech-

niques - MUTATION (TAICPART-MUTATION 2007), pp 169–175, DOI 10.1109/TAIC.PART.2007.38

Jafari O, Maurya P, Nagarkar P, Islam KM, Crushev C (2021) A survey on locality sensitive hashing algorithms and their applications. URL https://arxiv.org/abs/2102.08942, 2102.08942

Jia Y, Harman M (2011a) An analysis and survey of the development of mutation testing. IEEE Transactions on Software Engineering 37(5):649–678, DOI 10.1109/TSE.2010.62

Jia Y, Harman M (2011b) An analysis and survey of the development of mutation testing. IEEE Transactions on Software Engineering 37(5):649–678, DOI 10.1109/TSE.2010.62

Just R (2014) The major mutation framework: Efficient and scalable mutation analysis for java. In: Proceedings of the 2014 international symposium on software testing and analysis, pp 433–436

Just R, Schweiggert F, Kapfhammer GM (2011) Major: An efficient and extensible tool for mutation analysis in a java compiler. In: 2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011), pp 612–615, DOI 10.1109/ASE.2011.6100138

Just R, Kapfhammer GM, Schweiggert F (2012) Do redundant mutants affect the effectiveness and efficiency of mutation analysis? In: 2012 IEEE Fifth International Conference on Software Testing, Verification and Validation, pp 720–725, DOI 10.1109/ICST.2012.162

Just R, Jalali D, Inozemtseva L, Ernst MD, Holmes R, Fraser G (2014) Are mutants a valid substitute for real faults in software testing? In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, Association for Computing Machinery, New York, NY, USA, FSE 2014, p 654–665, DOI 10.1145/2635868.2635929, URL https://doi.org/10.1145/2635868.2635929

Kintis M, Papadakis M, Papadopoulos A, Valvis E, Malevris N (2016) Analysing and comparing the effectiveness of mutation testing tools: A manual study. In: 2016 IEEE 16th International Working Conference on Source Code Analysis and Manipulation (SCAM), pp 147–156, DOI 10.1109/SCAM.2016.28

Kintis M, Papadakis M, Jia Y, Malevris N, Le Traon Y, Harman M (2018) Detecting trivial mutant equivalences via compiler optimisations. IEEE Transactions on Software Engineering 44(4):308–333, DOI 10.1109/TSE.2017.2684805

Kurtz B, Ammann P, Offutt J, Kurtz M (2016) Are we there yet? how redundant and equivalent mutants affect determination of test completeness. In: 2016 IEEE Ninth International Conference on Software Testing, Verification and Validation Workshops (ICSTW), pp 142–151, DOI 10.1109/ICSTW.2016.41

Laurent T, Ventresque A (2019) Pit-hom: an extension of pitest for higher order mutation analysis. In: 2019 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), pp 83–89, DOI 10.1109/ICSTW.2019.00036

Laurent T, Gaffney S, Ventresque A (2022) Re-visiting the coupling between mutants and real faults with defects4j 2.0. In: 2022 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), pp 189–198, DOI 10.1109/ICSTW55395.2022.00042

Li Z, Shin D (2024) Mutation-based consistency testing for evaluating the code understanding capability of llms. In: Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI, Association for Computing Machinery, New York, NY, USA, CAIN '24, p 150–159, DOI 10.1145/3644815.3644946, URL https://doi.org/10.1145/3644815.3644946

Lin D, Koppel J, Chen A, Solar-Lezama A (2017) Quixbugs: a multi-lingual program repair benchmark set based on the quixey challenge. In: Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity, Association for Computing Machinery, New York, NY, USA, SPLASH Companion 2017, p 55–56, DOI 10.1145/3135932.3135941, URL https://doi.org/10.1145/3135932.3135941

Ma YS, Offutt J, Kwon YR (2006) Mujava: a mutation system for java. In: Proceedings of the 28th international conference on Software engineering, pp 827–830

Madeyski L, Radyk N (2010) Judy–a mutation testing tool for java. IET software 4(1):32–42

Madeyski L, Orzeszyna W, Torkar R, Józala M (2014) Overcoming the equivalent mutant problem: A systematic literature review and a comparative experiment of second order mutation. IEEE Transactions on Software Engineering 40(1):23–42, DOI 10.1109/TSE.2013.44

Malkov YA, Yashunin DA (2018) Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. URL https://arxiv.org/abs/1603.09320, 1603.09320

Martinez M, Falleri JR, Monperrus M (2023) Hyperparameter optimization for ast differencing. IEEE Transactions on Software Engineering 49(10):4814–4828, DOI 10.1109/TSE.2023.3315935

Mathur A (1991) Performance, effectiveness, and reliability issues in software testing. In: [1991] Proceedings The Fifteenth Annual International Computer Software & Applications Conference, pp 604–605, DOI 10.1109/CMPSAC.1991.170248

Nguyen QV, Madeyski L (2014) Problems of mutation testing and higher order mutation testing. In: Advanced Computational Methods for Knowledge Engineering: Proceedings of the 2nd International Conference on Computer Science, Applied Mathematics and Applications (ICCSAMA 2014), Springer, pp 157–172

Offutt AJ, Pan J (1996) Detecting equivalent mutants and the feasible path problem. In: Proceedings of 11th Annual Conference on Computer Assurance. COMPASS'96, IEEE, pp 224–236

Offutt AJ, Pan J (1997) Automatically detecting equivalent mutants and infeasible paths. Software testing, verification and reliability 7(3):165–192

Ojdanic M, Garg A, Khanfir A, Degiovanni R, Papadakis M, Le Traon Y (2023a) Syntactic versus semantic similarity of artificial and real faults in mutation testing studies. IEEE Transactions on Software Engineering 49(7):3922–3938, DOI 10.1109/TSE.2023.3277564

Ojdanic M, Garg A, Khanfir A, Degiovanni R, Papadakis M, Le Traon Y (2023b) Syntactic versus semantic similarity of artificial and real faults in mutation testing studies. IEEE Transactions on Software Engineering 49(7):3922–3938, DOI 10.1109/TSE.2023.3277564

Papadakis M, Delamaro M, Le Traon Y (2014) Mitigating the effects of equivalent mutants with mutant classification strategies. Science of Computer Programming 95:298–319

Papadakis M, Jia Y, Harman M, Le Traon Y (2015) Trivial compiler equivalence: A large scale empirical study of a simple, fast and effective equivalent mutant detection technique. In: 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering, vol 1, pp 936–946, DOI 10.1109/ICSE.2015.103

Papadakis M, Shin D, Yoo S, Bae DH (2018) Are mutation scores correlated with real fault detection? a large scale empirical study on the relationship between mutants and real faults. In: Proceedings of the 40th International Conference on Software Engineering, Association for Computing Machinery, New York, NY, USA, ICSE '18, p 537–548, DOI 10.1145/3180155.3180183, URL https://doi.org/10.1145/3180155.3180183

Papadakis M, Kintis M, Zhang J, Jia Y, Traon Y, Harman M (2019a) Chapter six-mutation testing advances: An analysis and survey. volume 112 of advances in computers

Papadakis M, Kintis M, Zhang J, Jia Y, Traon YL, Harman M (2019b) Chapter six - mutation testing advances: An analysis and survey. Advances in Computers, vol 112, Elsevier, pp 275–378, DOI https://doi.org/10.1016/bs.adcom.2018.03.015, URL https://www.sciencedirect.com/science/article/pii/S0065245818300305

Papineni K, Roukos S, Ward T, Zhu WJ (2002) Bleu: a method for automatic evaluation of machine translation. In: Proceedings of the 40th Annual Meeting on Association for Computational Linguistics, Association for Computational Linguistics, USA, ACL '02, p 311–318, DOI 10.3115/1073083.1073135, URL https://doi.org/10.3115/1073083.1073135

Schuler D, Zeller A (2010) (un-) covering equivalent mutants. In: 2010 Third International Conference on Software Testing, Verification and Validation, IEEE, pp 45–54

Soudani H, Kanoulas E, Hasibi F (2024) Fine tuning vs. retrieval augmented generation for less popular knowledge. In: Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region, Association for Computing Machinery, New York, NY, USA, SIGIR-AP 2024, p 12–22, DOI 10.1145/3673791.3698415, URL https://doi.org/10.1145/3673791.3698415

Tian Z, Chen J, Zhu Q, Yang J, Zhang L (2023) Learning to construct better mutation faults. In: Proceedings of the 37th IEEE/ACM International

Conference on Automated Software Engineering, Association for Computing Machinery, New York, NY, USA, ASE '22, DOI 10.1145/3551349.3556949, URL https://doi.org/10.1145/3551349.3556949

Tian Z, Shu H, Wang D, Cao X, Kamei Y, Chen J (2024) Large language models for equivalent mutant detection: How far are we? In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, Association for Computing Machinery, New York, NY, USA, ISSTA 2024, p 1733–1745, DOI 10.1145/3650212.3680395, URL https://doi.org/10.1145/3650212.3680395

Tip F, Bell J, Schaefer M (2025) Llmorpheus: Mutation testing using large language models. URL https://arxiv.org/abs/2404.09952, 2404.09952

Tufano M, Watson C, Bavota G, Di Penta M, White M, Poshyvanyk D (2019) Learning how to mutate source code from bug-fixes. In: 2019 IEEE International Conference on Software Maintenance and Evolution (ICSME), pp 301–312, DOI 10.1109/ICSME.2019.00046

Tufano M, Kimko J, Wang S, Watson C, Bavota G, Di Penta M, Poshyvanyk D (2020) Deepmutation: a neural mutation tool. Association for Computing Machinery, New York, NY, USA, ICSE '20, p 29–32, DOI 10.1145/3377812.3382146, URL https://doi.org/10.1145/3377812.3382146

Wang B, Xiong Y, Shi Y, Zhang L, Hao D (2017) Faster mutation analysis via equivalence modulo states. In: Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis, Association for Computing Machinery, New York, NY, USA, ISSTA 2017, p 295–306, DOI 10.1145/3092703.3092714, URL https://doi.org/10.1145/3092703.3092714

Wang B, Chen M, Lin Y, Papadakis M, Zhang JM (2024a) An exploratory study on using large language models for mutation testing. URL https://arxiv.org/abs/2406.09843, 2406.09843

Wang J, Huang Y, Chen C, Liu Z, Wang S, Wang Q (2024b) Software testing with large language models: Survey, landscape, and vision. IEEE Trans Softw Eng 50(4):911–936, DOI 10.1109/TSE.2024.3368208, URL https://doi.org/10.1109/TSE.2024.3368208

Wei J, Bosma M, Zhao VY, Guu K, Yu AW, Lester B, Du N, Dai AM, Le QV (2022) Finetuned language models are zero-shot learners. URL https://arxiv.org/abs/2109.01652, 2109.01652

Wei J, Wang X, Schuurmans D, Bosma M, Ichter B, Xia F, Chi E, Le Q, Zhou D (2023) Chain-of-thought prompting elicits reasoning in large language models. URL https://arxiv.org/abs/2201.11903, 2201.11903

Weimer W, Fry ZP, Forrest S (2013) Leveraging program equivalence for adaptive program repair: Models and first results. In: 2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp 356–366, DOI 10.1109/ASE.2013.6693094

White J, Fu Q, Hays S, Sandborn M, Olea C, Gilbert H, Elnashar A, Spencer-Smith J, Schmidt DC (2023) A prompt pattern catalog to enhance prompt engineering with chatgpt. URL https://arxiv.org/abs/2302.11382, 2302.11382

Wong CP, Meinicke J, Chen L, Diniz JaP, Kästner C, Figueiredo E (2020) Efficiently finding higher-order mutants. In: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Association for Computing Machinery, New York, NY, USA, ESEC/FSE 2020, p 1165–1177, DOI 10.1145/3368089.3409713, URL https://doi.org/10.1145/3368089.3409713

Wong WE (1993) On mutation and data flow. Purdue University

Wright CJ, Kapfhammer GM, McMinn P (2014) The impact of equivalent, redundant, and quasi mutants on database schema mutation analysis. In: Proceedings of the 14th International Conference on Quality Software

Xiao YA, Yang C, Wang B, Xiong Y (2023) Expressapr: Efficient patch validation for java automated program repair systems. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp 2038–2041, DOI 10.1109/ASE56229.2023.00012

Yao X, Harman M, Jia Y (2014) A study of equivalent and stubborn mutation operators using human analysis of equivalence. In: Proceedings of the 36th International Conference on Software Engineering, Association for Computing Machinery, New York, NY, USA, ICSE 2014, p 919–930, DOI 10.1145/2568225.2568265, URL https://doi.org/10.1145/2568225.2568265

Yi J, Tan SH, Mechtaev S, Böhme M, Roychoudhury A (2018) [journal first] a correlation study between automated program repair and test-suite metrics. In: 2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE), pp 24–24, DOI 10.1145/3180155.3182517

Zhang J, Zhang L, Harman M, Hao D, Jia Y, Zhang L (2019) Predictive mutation testing. IEEE Transactions on Software Engineering 45(9):898–918, DOI 10.1109/TSE.2018.2809496